

Keycloak & Spring Security

Le guide complet, expliqué et illustré par le code

Formateur : Haithem Mihoubi

Pile : Spring Boot 3 · Spring Security 6 · Keycloak 24 · Angular 17

Niveau : intermédiaire → avancé

Pré-requis : Java 17+, bases de Spring Boot et des API REST

Projet support : QuickBite (resource server + façade /auth + front)

Sommaire

1. Pourquoi déléguer l'authentification ?	3
1.1 Le problème	3
1.2 La solution : un serveur d'identité (IAM)	3
1.3 Deux approches comparées	3
2. OAuth2 et OpenID Connect en profondeur	4
2.1 OAuth2 : l'autorisation déléguée	4
2.2 OpenID Connect : l'authentification	4
2.3 Les flows (scénarios d'obtention de token)	4
2.4 Anatomie d'un access token Keycloak	5
3. Keycloak en profondeur	6
3.1 Les objets de Keycloak	6
3.2 Types de clients	6
3.3 Le realm QuickBite (fichier d'import)	6
3.4 Lancer Keycloak en Docker	8
4. Spring Security 6 : l'API en resource server	9
4.1 Le principe	9
4.2 Dépendances Maven	9
4.3 Configuration de l'issuer	10
4.4 La SecurityConfig complète	11
4.5 Autorisation fine avec @PreAuthorize	12
5. Lire l'identité dans un contrôleur	13
6. La façade d'authentification (architecture entreprise)	14
6.1 Paramètres	14
6.2 Login / Refresh / Logout (client public)	14
6.3 Signup via l'Admin API + service account	15
6.4 Le contrôleur	16
7. Le front Angular	17
7.1 Service d'authentification	17
7.2 Intercepteur avec refresh automatique	18
8. Toute la pile en Docker Compose	19
9. Tester de bout en bout	20
10. Sécurité avancée & production	21
11. Dépannage (problèmes réels rencontrés)	22
12. Récapitulatif	23

1. Pourquoi déléguer l'authentification ?

1.1 Le problème

Chaque application a besoin de répondre à deux questions : **qui es-tu ?** (authentification) et **as-tu le droit ?** (autorisation). Si chaque application gère elle-même les comptes, les mots de passe, les sessions, le « mot de passe oublié », la double authentification... on se retrouve avec :

- du **code de sécurité dupliqué** dans chaque service (et donc des failles dupliquées) ;
- des **mots de passe stockés partout** (surface d'attaque énorme) ;
- **aucun SSO** (l'utilisateur se reconnecte sur chaque application) ;
- une gestion des utilisateurs **éclatée** et difficile à auditer.

1.2 La solution : un serveur d'identité (IAM)

On **centralise l'identité** dans un serveur dédié (un *Identity and Access Management*). Les applications ne gèrent plus les mots de passe : elles **font confiance** à des **tokens** signés émis par ce serveur. C'est le rôle de **Keycloak**.

Idée directrice

L'application ne sait plus « vérifier un mot de passe ». Elle sait seulement **vérifier la signature d'un token** émis par une autorité de confiance (Keycloak) et **lire les rôles** qu'il contient. L'authentification est **déléguée** ; l'application ne fait plus que de l'**autorisation**.

1.3 Deux approches comparées

Dans la formation, on construit deux variantes de la même API « QuickBite » :

	JWT « maison »	Keycloak (OIDC)
Qui gère les comptes ?	L'application (base + BCrypt)	Keycloak
Qui émet les tokens ?	L'application (signe en HMAC)	Keycloak (signe en RSA)
SSO / fédération / MFA	À coder soi-même	Fourni
Rotation des clés	À coder	Automatique (JWKS)
Idéal pour	Comprendre la mécanique	Production / entreprise

Ce cours se concentre sur la variante **Keycloak**, qui est l'approche **entreprise**, tout en réutilisant les fondamentaux de Spring Security.

2. OAuth2 et OpenID Connect en profondeur

Keycloak implémente deux protocoles standards. Il faut les comprendre avant de coder.

2.1 OAuth2 : l'autorisation déléguée

OAuth2 répond à : « cette application a-t-elle le droit d'accéder à telles ressources **en mon nom** ? » — sans jamais lui communiquer mon mot de passe. Il définit **quatre acteurs** :

Acteur	Rôle	Dans QuickBite
Resource Owner	L'utilisateur propriétaire des données	La personne qui se connecte
Client	L'application qui demande l'accès	Le front Angular
Authorization Server	Émet les tokens après authentification	Keycloak
Resource Server	Héberge les ressources protégées	L'API Spring Boot

2.2 OpenID Connect : l'authentification

OAuth2 seul ne dit pas formellement *qui* est l'utilisateur. **OpenID Connect (OIDC)** est une **couche d'identité** par-dessus OAuth2 : il ajoute un **ID token** (qui décrit l'utilisateur) et un endpoint de **découverte** standard (`/.well-known/openid-configuration`).

ID token vs Access token

L'**ID token** prouve *qui est l'utilisateur* (claims `name` , `email` ...) et est destiné au **client** (le front). L'**access token** autorise l'accès à une **API** et est destiné au **resource server**. C'est l'**access token** qu'on envoie dans l'en-tête `Authorization: Bearer ...` de chaque appel à l'API.

2.3 Les flows (scénarios d'obtention de token)

Authorization Code + PKCE (recommandé pour web / SPA / mobile)

```
Utilisateur → Client → Keycloak (page de login)
                                   | (login + consentement)
                                   |
← code —————
Client → échange (code + code_verifier PKCE) → Keycloak
       ← access_token (+ id_token + refresh_token) —
Client → appelle l'API avec l'access_token → Resource Server
```

Flow	Usage	Statut
Authorization Code + PKCE	Web, SPA, mobile	Recommandé
Client Credentials	Machine-à-machine (pas d'utilisateur)	OK pour les services
Password (Direct Grant)	Scripts / tests / clients de confiance	À éviter en prod
~~Implicit~~	Ancien SPA	Déconseillé

💡 PKCE en deux phrases

Pour les clients « publics » (SPA, mobile) qui ne peuvent pas garder de secret, PKCE protège l'échange du code : le client invente un secret aléatoire (`code_verifier`), n'envoie d'abord que son empreinte (`code_challenge`), puis prouve à la fin qu'il détient l'original. Un code intercepté devient inutilisable.

2.4 Anatomie d'un access token Keycloak

Un token Keycloak est un **JWT signé en RSA**. Décodé (sur jwt.io), son payload ressemble à :

```
{
  "exp": 1735738200,
  "iss": "http://keycloak:8080/realms/quickbite",
  "preferred_username": "sarah",
  "realm_access": {
    "roles": ["client", "default-roles-quickbite"]
  },
  "scope": "openid profile email"
}
```

Deux claims sont cruciaux pour Spring : `iss` (l'émetteur, qu'on va valider) et `realm_access.roles` (les rôles, qu'on va mapper en autorités).

❗ Signé ≠ chiffré

Comme tout JWT, le payload est seulement encodé en base64 : **lisible**. La signature RSA garantit qu'il n'a pas été **modifié** et qu'il vient bien de Keycloak — pas qu'il est secret. N'y mettez jamais de donnée sensible.

3. Keycloak en profondeur

3.1 Les objets de Keycloak

Objet	Définition
Realm	Un espace totalelement isolé : ses utilisateurs, clients, rôles, clés. On crée un realm par « organisation logique » (ex. <code>quickbite</code>).
Client	Une application enregistrée qui demande des tokens (le front, l'API, un service).
Realm role / Client role	Un rôle global au realm (<code>admin</code> , <code>client</code>) ou propre à un client.
User	Un compte (identifiants, attributs, rôles, groupes).
Group	Un regroupement d'utilisateurs qui héritent de rôles.
Service account	Un « compte » rattaché à un client confidentiel, pour les appels machine-à-machine.
Mapper	Règle qui injecte une donnée (rôle, attribut) dans le token.

3.2 Types de clients

- **Public** : ne peut pas garder de secret (SPA, mobile). Utilise Authorization Code + PKCE.
- **Confidential** : possède un *client secret* (backend). Peut utiliser Client Credentials.
- **Bearer-only** (historique) : ne fait que valider des tokens (un resource server pur).

Dans QuickBite on a **deux clients** :

- `quickbite` — **public**, pour le front (login) et les tests (direct grant) ;
- `quickbite-backend` — **confidential** avec **service account**, pour que l'API crée des comptes (signup) via l'Admin API.

3.3 Le realm QuickBite (fichier d'import)

Keycloak peut **importer un realm au démarrage** (`--import-realm`). Voici le fichier complet utilisé — il définit les rôles, les clients et les utilisateurs :

```

{
  "realm": "quickbite",
  "enabled": true,
  "sslRequired": "none",
  "accessTokenLifespan": 900,
  "requiredActions": [
    { "alias": "VERIFY_PROFILE", "name": "Verify Profile",
      "providerId": "VERIFY_PROFILE", "enabled": false, "defaultAction": false }
  ],
  "roles": {
    "realm": [
      { "name": "admin", "description": "Administrateur QuickBite" },
      { "name": "client", "description": "Client QuickBite" }
    ]
  },
  "clients": [
    {
      "clientId": "quickbite",
      "publicClient": true,
      "standardFlowEnabled": true,
      "directAccessGrantsEnabled": true,
      "redirectUris": ["http://localhost:4300/*", "*"],
      "webOrigins": ["*"],
      "attributes": { "pkce.code.challenge.method": "S256" }
    },
    {
      "clientId": "quickbite-backend",
      "publicClient": false,
      "secret": "backend-secret",
      "serviceAccountsEnabled": true,
      "standardFlowEnabled": false,
      "directAccessGrantsEnabled": false
    }
  ],
  "users": [
    {
      "username": "admin", "enabled": true, "emailVerified": true,
      "firstName": "Admin", "lastName": "QuickBite", "email": "admin@quickbite.local",
      "credentials": [{ "type": "password", "value": "admin", "temporary": false }],
      "realmRoles": ["admin", "client"]
    },
    {
      "username": "client", "enabled": true, "emailVerified": true,
      "firstName": "Client", "lastName": "QuickBite", "email": "client@quickbite.local",
      "credentials": [{ "type": "password", "value": "client", "temporary": false }],
      "realmRoles": ["client"]
    },
    {
      "username": "service-account-quickbite-backend", "enabled": true,
      "serviceAccountClientId": "quickbite-backend",
      "clientRoles": { "realm-management": ["manage-users", "view-users", "query-users", "view-realm"] }
    }
  ]
}

```

⚠ Le service account et ses rôles

Pour que l'API puisse créer des comptes, son service account doit avoir les rôles du client interne `realm-management` : `manage-users` (créer/modifier), et `view-realm` (lire la définition d'un rôle avant de l'assigner). Sans `view-realm`, l'assignation du rôle échoue en **403** — une erreur très fréquente.

3.4 Lancer Keycloak en Docker

```
docker run -p 8080:8080 \
  -e KEYCLOAK_ADMIN=admin -e KEYCLOAK_ADMIN_PASSWORD=admin \
  -v ./keycloak/realm-quickbite.json:/opt/keycloak/data/import/realm.json \
  quay.io/keycloak/keycloak:24.0 start-dev --import-realm
```

En production on utilise `start` (mode optimisé) avec une **base PostgreSQL** dédiée et un **hostname fixe** (voir le `docker-compose` au chapitre 8).

4. Spring Security 6 : l'API en resource server

4.1 Le principe

Spring Security s'insère comme une **chaîne de filtres** devant l'application. Pour un resource server OIDC, le filtre clé est le **BearerTokenAuthenticationFilter** : il extrait le token **Bearer**, le **valide** (signature + expiration + émetteur) grâce au **JWKS** publié par Keycloak, puis construit l'**Authentication**.

```
Requête (Bearer ...) → BearerTokenAuthenticationFilter
    | récupère la clé publique via /realms/quickbite/protocol/openid-connect/certs (JWKS)
    | valide signature + exp + iss
    ▼
JwtAuthenticationConverter — mappe realm_access.roles -> ROLE_*
    ▼
SecurityContext → AuthorizationManager (@PreAuthorize) → Contrôleur
```

JWKS : la rotation de clés « gratuite »

Keycloak publie ses clés **publiques** à l'URL `.../certs` (le JWKS). Spring les récupère et les met en cache. Quand Keycloak **change de clé** (rotation), Spring récupère automatiquement la nouvelle via le **kid** du token. Vous n'avez **rien à coder** pour la rotation des clés — un énorme avantage par rapport au JWT maison.

4.2 Dépendances Maven

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

4.3 Configuration de l'issuer

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          # Spring lit l'OIDC discovery de l'issuer pour trouver le JWKS,
          # et vérifie que le claim "iss" du token correspond exactement.
          issuer-uri: ${ISSUER_URI:http://keycloak:8080/realms/quickbite}
```

⚠ Le piège de l'issuer en Docker

Le claim `iss` du token doit correspondre **exactement** à `issuer-uri`. Si le navigateur obtient le token via `http://localhost:8081` mais que l'API valide `http://keycloak:8080`, la validation échoue (401). Deux solutions : (1) un **hostname Keycloak fixe** identique pour tous, ou (2) une **façade** côté API qui proxy vers Keycloak en interne (voir chapitre 6) — c'est l'approche retenue dans QuickBite.

4.4 La SecurityConfig complète

```

@Configuration
@EnableWebSecurity
@EnableMethodSecurity // active @PreAuthorize
public class SecurityConfig {

    @Value("${quickbite.cors.allowed-origins}")
    private List<String> allowedOrigins;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .cors(c -> c.configurationSource(corsSource()))
            .csrf(csrf -> csrf.disable()) // API par token
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/health", "/actuator/health/**").permitAll()
                .requestMatchers("/auth/**").permitAll() // façade publique
                .requestMatchers("/admin/**").hasRole("admin")
                .anyRequest().authenticated())
            .oauth2ResourceServer(oauth2 -> oauth2.jwt(jwt ->
                jwt.jwtAuthenticationConverter(jwtAuthConverter())));
        return http.build();
    }

    // --- Mapping des rôles Keycloak (realm_access.roles) -> autorités ROLE_* ---
    private JwtAuthenticationConverter jwtAuthConverter() {
        JwtAuthenticationConverter conv = new JwtAuthenticationConverter();
        conv.setPrincipalClaimName("preferred_username");
        conv.setJwtGrantedAuthoritiesConverter(SecurityConfig::extractRoles);
        return conv;
    }

    @SuppressWarnings("unchecked")
    private static Collection<GrantedAuthority> extractRoles(Jwt jwt) {
        Map<String, Object> realmAccess = jwt.getClaim("realm_access");
        if (realmAccess == null) return List.of();
        Collection<String> roles = (Collection<String>) realmAccess.getOrDefault("roles", List.of());
        return roles.stream()
            .map(r -> (GrantedAuthority) new SimpleGrantedAuthority("ROLE_" + r))
            .toList();
    }

    @Bean
    public CorsConfigurationSource corsSource() {
        CorsConfiguration c = new CorsConfiguration();
        c.setAllowedOrigins(allowedOrigins);
        c.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
        c.setAllowedHeaders(List.of("Authorization", "Content-Type"));
        c.setAllowCredentials(true);
        UrlBasedCorsConfigurationSource src = new UrlBasedCorsConfigurationSource();
        src.registerCorsConfiguration("/**", c);
        return src;
    }
}

```

! Le préfixe ROLE_ et la casse

Keycloak nomme ses rôles en minuscules (`admin` , `client`). On les mappe en `ROLE_admin` / `ROLE_client` . Ensuite `hasRole('admin')` cherche l'autorité `ROLE_admin` . Respectez la même casse partout, sinon : accès refusé (403) sans raison apparente.

4.5 Autorisation fine avec @PreAuthorize

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    private final OrderRepository repo;
    public OrderController(OrderRepository repo) { this.repo = repo; }

    public record CreateOrder(@NotBlank String item, @Positive double price) { }

    @GetMapping
    @PreAuthorize("isAuthenticated()")
    public List<OrderEntity> myOrders(Authentication auth) {
        return repo.findByOwner(auth.getName()); // auth.getName() = preferred_username
    }

    @PostMapping
    @PreAuthorize("hasRole('client') or hasRole('admin')")
    @ResponseStatus(HttpStatus.CREATED)
    public OrderEntity create(@RequestBody CreateOrder body, Authentication auth) {
        return repo.save(new OrderEntity(body.item(), body.price(), auth.getName()));
    }

    @DeleteMapping("/{id}")
    @PreAuthorize("hasRole('admin')")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) { repo.deleteById(id); }
}
```

La console d'administration est verrouillée à deux niveaux (URL + méthode) :

```
@RestController
@RequestMapping("/admin")
@PreAuthorize("hasRole('admin')")
public class AdminController {
    private final OrderRepository orders;
    public AdminController(OrderRepository orders) { this.orders = orders; }

    @GetMapping("/summary")
    public Map<String, Object> summary() {
        return Map.of(
            "totalOrders", orders.count(),
            "totalRevenue", orders.findAll().stream().mapToDouble(OrderEntity::getPrice).sum());
    }
}
```

5. Lire l'identité dans un contrôleur

Une fois le token validé, Spring expose l'utilisateur. Trois façons de le lire :

```
// 1) Via Authentication (nom = preferred_username, autorités = ROLE_*)
@GetMapping("/me")
public Map<String, Object> me(Authentication auth) {
    return Map.of("username", auth.getName(), "authorities", auth.getAuthorities());
}

// 2) Via l'objet Jwt complet (tous les claims)
@GetMapping("/claims")
public Map<String, Object> claims(@AuthenticationPrincipal Jwt jwt) {
    return Map.of("email", jwt.getClaimAsString("email"),
        "issuer", jwt.getIssuer().toString());
}
```

Codes de réponse à connaître : **401** (pas de token / token invalide), **403** (token valide mais rôle insuffisant).

6. La façade d'authentification (architecture entreprise)

Plutôt que d'exposer Keycloak directement au front, on place dans l'API une **façade** `/auth` : `register`, `login`, `refresh`, `logout`. Avantages :

- le front a une **API simple et stable**, sans dépendre des URLs Keycloak ;
- on **résout le problème d'issuer** (l'API parle à Keycloak en interne) ;
- on peut **enrichir** le signup (validation métier, envoi d'email...).

6.1 Paramètres

```
keycloak:
  base-url: ${KEYCLOAK_BASE_URL:http://keycloak:8080}
  realm: quickbite
  public-client-id: quickbite
  backend-client-id: quickbite-backend
  backend-client-secret: ${KEYCLOAK_BACKEND_SECRET:backend-secret}
  default-role: client
```

6.2 Login / Refresh / Logout (client public)

```
public Map<String, Object> login(String username, String password) {
    MultiValueMap<String, String> form = new LinkedMultiValueMap<>();
    form.add("client_id", props.getPublicClientId());
    form.add("grant_type", "password");
    form.add("username", username);
    form.add("password", password);
    return token(form, "Identifiants invalides");
}

public Map<String, Object> refresh(String refreshToken) {
    MultiValueMap<String, String> form = new LinkedMultiValueMap<>();
    form.add("client_id", props.getPublicClientId());
    form.add("grant_type", "refresh_token");
    form.add("refresh_token", refreshToken);
    return token(form, "Refresh token invalide ou expiré");
}

private Map<String, Object> token(MultiValueMap<String, String> form, String errMsg) {
    try {
        return http.post().uri(props.tokenUrl())
            .contentType(MediaType.APPLICATION_FORM_URLENCODED)
            .body(form).retrieve().body(Map.class);
    } catch (Exception e) {
        throw new ResponseStatusException(HttpStatus.UNAUTHORIZED, errMsg);
    }
}
```

6.3 Signup via l'Admin API + service account

Le **signup** est le cœur « entreprise » : l'API obtient un token de service (grant `client_credentials`), puis crée l'utilisateur et lui assigne le rôle `client` — sans aucun identifiant admin en dur.

```
public Map<String, Object> register(RegisterRequest req) {
    String adminToken = serviceToken(); // client_credentials

    Map<String, Object> userRep = Map.of(
        "username", req.username(),
        "email", req.email(),
        "firstName", firstNameOrDefault(req),
        "lastName", lastNameOrDefault(req),
        "enabled", true, "emailVerified", true,
        "credentials", List.of(Map.of(
            "type", "password", "value", req.password(), "temporary", false)));

    ResponseEntity<Void> created = http.post().uri(props.adminUsersUrl())
        .header("Authorization", "Bearer " + adminToken)
        .contentType(MediaType.APPLICATION_JSON)
        .body(userRep).retrieve().toBodilessEntity();

    String userId = extractId(created.getHeaders().getLocation());
    assignRealmRole(adminToken, userId, props.getDefaultRole());
    return Map.of("id", userId, "username", req.username(), "role", props.getDefaultRole());
}

private String serviceToken() {
    MultiValueMap<String, String> form = new LinkedMultiValueMap<>();
    form.add("client_id", props.getBackendClientId());
    form.add("client_secret", props.getBackendClientSecret());
    form.add("grant_type", "client_credentials");
    Map<String, Object> resp = http.post().uri(props.tokenUrl())
        .contentType(MediaType.APPLICATION_FORM_URLENCODED)
        .body(form).retrieve().body(Map.class);
    return (String) resp.get("access_token");
}

private void assignRealmRole(String adminToken, String userId, String roleName) {
    Map<String, Object> role = http.get().uri(props.adminRoleUrl(roleName))
        .header("Authorization", "Bearer " + adminToken)
        .retrieve().body(Map.class); // nécessite view-realm
    http.post().uri(props.adminUsersUrl() + "/" + userId + "/role-mappings/realm")
        .header("Authorization", "Bearer " + adminToken)
        .contentType(MediaType.APPLICATION_JSON)
        .body(List.of(role)).retrieve().toBodilessEntity();
}
```

6.4 Le contrôleur

```
@RestController
@RequestMapping("/auth")
public class AuthController {
    private final KeycloakClient keycloak;
    public AuthController(KeycloakClient keycloak) { this.keycloak = keycloak; }

    @PostMapping("/register")
    @ResponseStatus(HttpStatus.CREATED)
    public Map<String, Object> register(@Valid @RequestBody RegisterRequest req) {
        return keycloak.register(req);
    }
    @PostMapping("/login")
    public Map<String, Object> login(@Valid @RequestBody LoginRequest req) {
        return keycloak.login(req.username(), req.password());
    }
    @PostMapping("/refresh")
    public Map<String, Object> refresh(@Valid @RequestBody RefreshRequest req) {
        return keycloak.refresh(req.refreshToken());
    }
    @PostMapping("/logout")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void logout(@Valid @RequestBody LogoutRequest req) {
        keycloak.logout(req.refreshToken());
    }
}
```


7. Le front Angular

Le front consomme la façade. Il **décode le JWT** (sans le vérifier — la vérification est faite par l'API) pour adapter l'interface au rôle.

7.1 Service d'authentification

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  private api = environment.apiUrl;
  readonly user = signal<CurrentUser | null>(this.decode(this.accessToken));
  readonly isAdmin = computed(() => this.user()?.roles.includes('admin') ?? false);

  login(username: string, password: string): Observable<TokenResponse> {
    return this.http.post<TokenResponse>(`${this.api}/auth/login`, { username, password })
      .pipe(tap((res) => this.store(res)));
  }

  refresh(): Observable<TokenResponse> {
    return this.http.post<TokenResponse>(`${this.api}/auth/refresh`,
      { refreshToken: this.refreshTokenValue }).pipe(tap((res) => this.store(res)));
  }

  // Décode realm_access.roles + preferred_username depuis le JWT Keycloak
  private decode(token: string | null): CurrentUser | null {
    if (!token) return null;
    try {
      const payload = JSON.parse(atob(token.split('.')[1]));
      return { username: payload.preferred_username, roles: payload?.realm_access?.roles ?? [] };
    } catch { return null; }
  }
}
```

7.2 Intercepteur avec refresh automatique

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const router = inject(Router);
  const isAuth = req.url.includes('/auth/');
  const token = auth.accessToken;
  const authReq = token && !isAuth
    ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } }) : req;

  return next(authReq).pipe(catchError((err: HttpErrorResponse) => {
    if (err.status === 401 && !isAuth && auth.refreshTokenValue) {
      return auth.refresh().pipe(
        // tente UN refresh puis rejoue
        switchMap(() => next(req.clone({
          setHeaders: { Authorization: `Bearer ${auth.accessToken}` } }))),
        catchError((e) => { auth.logout(); router.navigate(['/login']); return throwError(() => e); }));
    }
    return throwError(() => err);
  }));
};
```

8. Toute la pile en Docker Compose

```
services:
  keycloak-db:
    image: postgres:16-alpine
    environment: { POSTGRES_DB: keycloak, POSTGRES_USER: keycloak, POSTGRES_PASSWORD: keycloak }
    healthcheck: { test: ["CMD-SHELL", "pg_isready -U keycloak"], interval: 5s, retries: 20 }

  keycloak:
    image: quay.io/keycloak/keycloak:24.0
    command: ["start", "--import-realm"]
    environment:
      KC_DB: postgres
      KC_DB_URL: jdbc:postgresql://keycloak-db:5432/keycloak
      KC_DB_USERNAME: keycloak
      KC_DB_PASSWORD: keycloak
      KEYCLOAK_ADMIN: admin
      KEYCLOAK_ADMIN_PASSWORD: admin
      KC_HOSTNAME: keycloak # hostname STABLE -> issuer constant
      KC_HOSTNAME_PORT: "8080"
      KC_HOSTNAME_STRICT: "false"
      KC_HTTP_ENABLED: "true"
      KC_HEALTH_ENABLED: "true"
    volumes:
      - ./keycloak/realm-quickbite.json:/opt/keycloak/data/import/realm-quickbite.json:ro
    ports: ["8080:8080"]
    depends_on: { keycloak-db: { condition: service_healthy } }

  app-db:
    image: postgres:16-alpine
    environment: { POSTGRES_DB: quickbite, POSTGRES_USER: app, POSTGRES_PASSWORD: secret }
    healthcheck: { test: ["CMD-SHELL", "pg_isready -U app"], interval: 5s, retries: 20 }

  api:
    build: ./api
    environment:
      DB_URL: jdbc:postgresql://app-db:5432/quickbite
      ISSUER_URI: http://keycloak:8080/realms/quickbite
      KEYCLOAK_BASE_URL: http://keycloak:8080
      CORS_ORIGINS: http://localhost:4300
    ports: ["8082:8080"]
    depends_on:
      app-db: { condition: service_healthy }
      keycloak: { condition: service_healthy }
```

```
docker compose up -d --build # démarre toute la pile
```

9. Tester de bout en bout

```
API=http://localhost:8082

# 1) Signup
curl -s -X POST $API/auth/register -H "Content-Type: application/json" \
  -d '{"username":"sarah","email":"sarah@quickbite.local","password":"secret123"}'

# 2) Login -> récupérer le token
TOKEN=$(curl -s -X POST $API/auth/login -H "Content-Type: application/json" \
  -d '{"username":"sarah","password":"secret123"}' | jq -r .access_token)

# 3) Appels protégés
curl -s $API/me -H "Authorization: Bearer $TOKEN" # 200, rôles
curl -s -X POST $API/orders -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" -d '{"item":"Sushi","price":12}' # 201 (client)
curl -s -o /dev/null -w "%{http_code}\n" $API/admin/summary \
  -H "Authorization: Bearer $TOKEN" # 403 (pas admin)
```

Résultats attendus : **401** sans token, **200/201** avec token et bon rôle, **403** si rôle insuffisant.

10. Sécurité avancée & production

- **HTTPS partout** : terminaison TLS au reverse proxy / Ingress ; `KC_HOSTNAME` en `https://` .
Jamais de token en clair sur le réseau.
 - **Durées de vie** : access token court (5-15 min), refresh plus long ; activez la **rotation des refresh tokens** dans Keycloak.
 - **Rotation des clés de signature** : automatique via JWKS — rien à coder.
 - **Sessions & logout** : `POST /auth/logout` invalide la session Keycloak (backchannel) ; pour un logout global, utiliser l'endpoint OIDC `end-session` .
 - **Moindre privilège** : le service account n'a que `manage-users` + `view-realm` , pas les droits d'admin complets.
 - **CORS strict** : autorisez seulement les origines connues du front.
 - **Brute force** : activez la *Brute Force Detection* de Keycloak.
 - **OWASP** : le risque n°1 reste le *Broken Access Control* — vérifiez chaque endpoint (`@PreAuthorize`) et testez les cas 401/403.
-

11. Dépannage (problèmes réels rencontrés)

Symptôme	Cause	Solution
401 sur tous les appels	<code>iss</code> du token $\neq$ <code>issuer-uri</code>	Hostname Keycloak fixe ou façade qui parle à Keycloak en interne
401 « Account is not fully set up »	Action requise <i>Verify Profile</i> (profil incomplet)	Désactiver <code>VERIFY_PROFILE</code> dans le realm, ou fournir <code>firstName</code> / <code>lastName</code>
403 à l'assignation de rôle au signup	Service account sans <code>view-realm</code>	Ajouter <code>view-realm</code> aux rôles <code>realm-management</code>
403 alors que l'utilisateur a le rôle	Mauvais préfixe/casse (<code>ROLE_</code> ou majuscule)	Mapper en <code>ROLE_</code> + même casse que Keycloak
Realm non mis à jour	<code>--import-realm</code> ignore un realm existant	<code>docker compose down -v</code> puis <code>up</code> (base vierge)
Keycloak <code>unhealthy</code> au 1er démarrage	Temps de démarrage	Laisser ~60 s (<code>start_period</code>) ; vérifier les logs

12. Récapitulatif

- **Keycloak** centralise l'identité ; l'**API** ne fait que valider des tokens et autoriser — elle ne gère plus de mots de passe.
- **OAuth2** délègue l'autorisation, **OIDC** ajoute l'authentification (ID token).
- En Spring : `oauth2ResourceServer().jwt()` + un **converter** qui mappe `realm_access.roles` → `ROLE_*`, puis `@PreAuthorize` pour le RBAC.
- Une **façade** `/auth` (signup via Admin API + service account, login, refresh, logout) offre une API simple au front et règle le problème d'issuer.
- La **rotation des clés** et le **SSO** sont fournis « gratuitement » par Keycloak.
- Toute la pile (Keycloak + bases + API) se lance en **une commande** avec Docker Compose, et se teste de bout en bout (signup → login → RBAC).

✓ À emporter

La règle d'or : **authentification déléguée, autorisation locale**. L'API fait confiance aux tokens d'une autorité unique (Keycloak), valide leur signature via JWKS, et applique ses propres règles de rôles. C'est l'architecture de sécurité standard des systèmes d'entreprise modernes.